

Question # 6 (Points: 15)

Consider the following instruction that is not found in the RISC-V architecture:

Add to Memory

```
addm rd, Offset(rs)    // Reg[rd] = Reg[rd] +
                        // Mem[Offset + Reg[rs]]
```

We want to modify the RISC-V processor to support the above instruction. For parts (b) and (d) below, you can use a printed version of the figures in the book over which you can draw your suggested modifications (no need to draw the entire diagram from scratch).

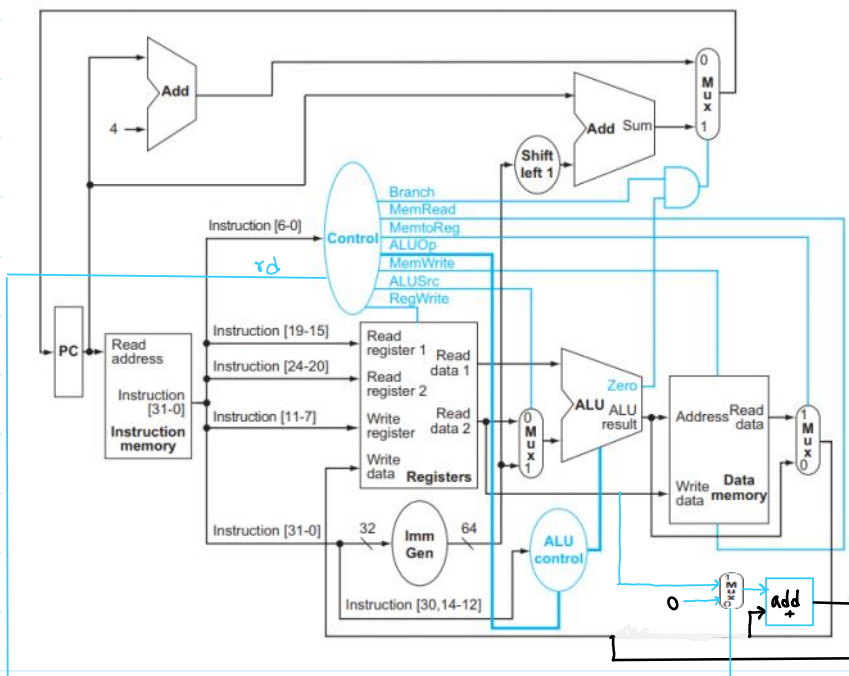
1. Suggest if any of the existing instruction formats is a good choice to encode the new instruction. If not, then propose a new instruction format. (2 marks)

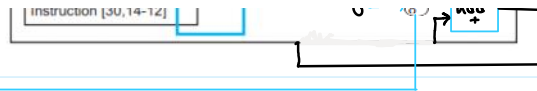
◦ as there is no such format which takes all of the rd, rs1, rs2, imm.

imm[9:3]	rs2	rs1	imm[2:0]	rd	opcode
[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]

- we will need a field for source register as we need to add value read from the memory to the value from a register, and store it in rd.
- so for addm, $rs = rs2 = rd$, $rs = rs1$, offset = imm

2. Modify the datapath and control signals of the single-cycle RISC-V processor (Figure 4.17 of the book) to execute the new instruction using the instruction format suggested in part A. Use the minimal amount of additional hardware and clock cycles/control states. Remember when adding new instructions, don't break the operation of the standard ones. (4 marks)



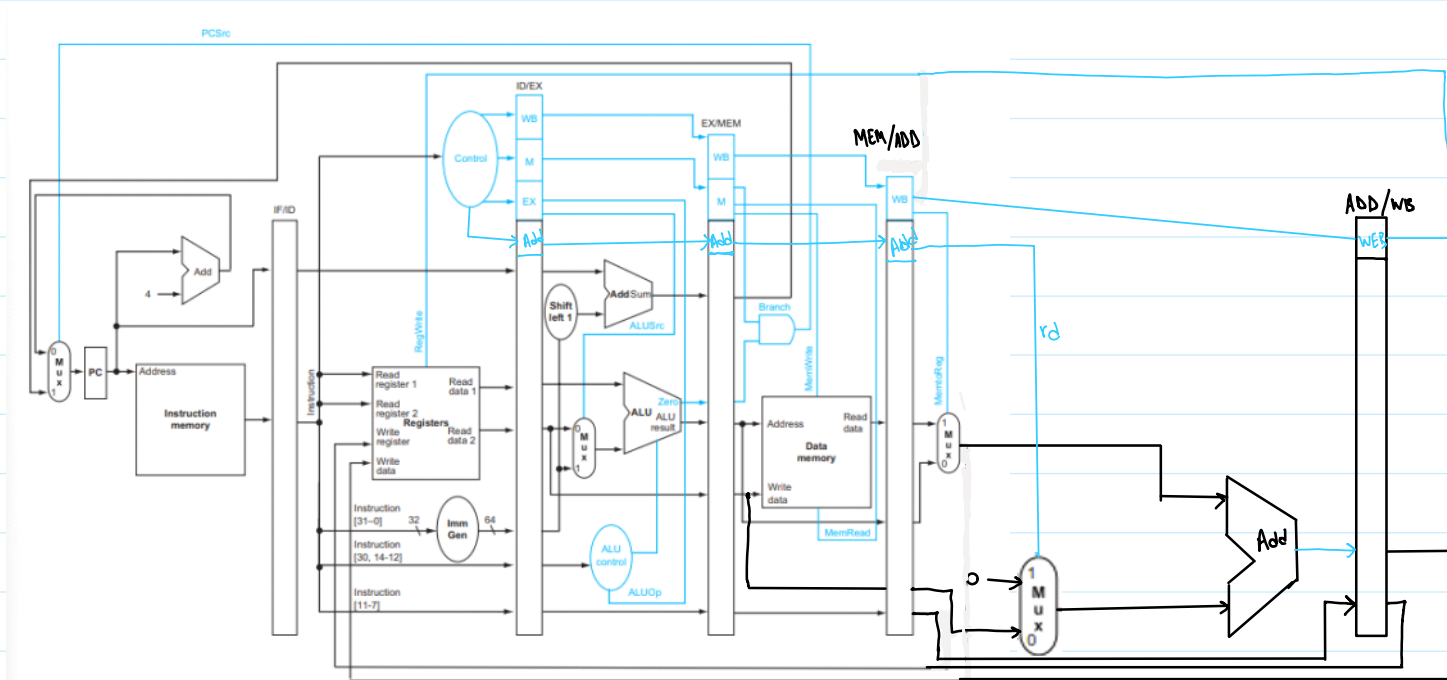


- here the rd signal will be generated by the control, so when instruction is addm, the rd signal will be 1 which will modify the data of write data, if wrd=0, then no modifications happens.

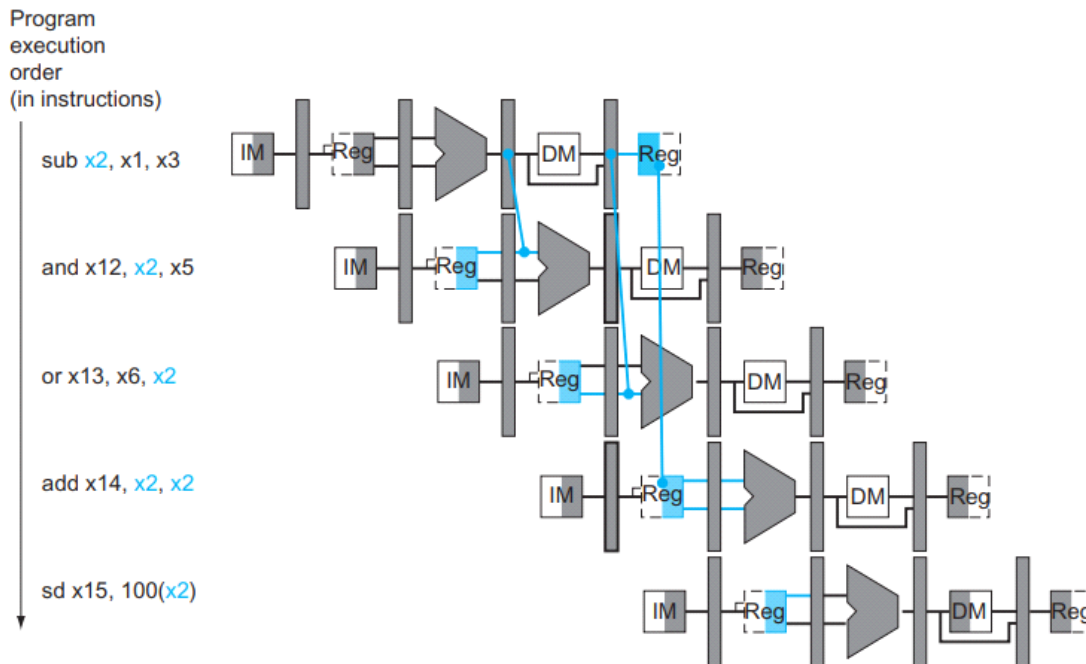
3. Discuss the effect of the modification in part (B) on the latency of single-cycle non-pipelined CPU. (2 marks)

- it will increase the latency for the critical path, because now the most longest path goes through an adder. The clock cycle will be increased

4. Discuss if the suggested modification in part (B) should be handled by increasing/decreasing the number of pipelining stages that were discussed in class. Draw a pipelined version of the new processor similar to Figure 4.49 of the book. (4 marks)



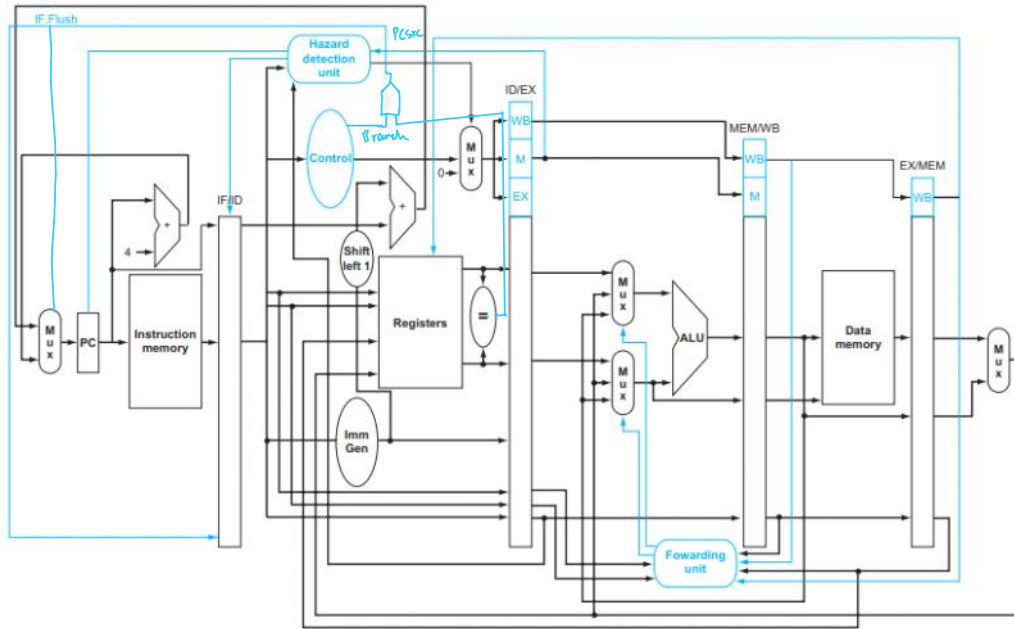
5. Discuss if any new types of data hazards are introduced due to the new instruction? If yes, can they be mitigated through forwarding? Use a multi-cycle pipeline diagram like Figure 4.51 of the book to illustrate the new forwarding paths. (3 marks)



- In the modified pipeline the result of a load instruction is written back later than before, so it introduces additional data hazards, mainly due to increased latency in producing results.

Question # 7 (Points: 10)

The pipelined processor's performance might be better if branches take place during the Decode stage rather than the Execute stage. Show how to modify the pipelined processor to move the branch logic to the Decode stage. How do the stall, flush, and forwarding signals change? You can use a printed version of the figures in the book over which you can draw your suggested modifications.



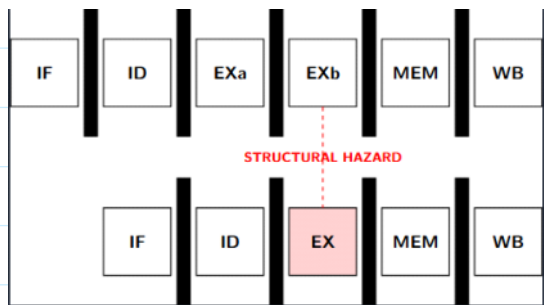
- To implement this in ID stage, we have to add the following hardware to the section between IF/ID register and the ID/EX register.
- Target adder:
This adder will take the current PC and the immediate to calculate the target address.
- Equality comparator:
A simple comparator immediately after register file to check if $rs1 = rs2$.
for beq instruction
- Control move:
A PCsrc signal that decides between $PC+4$ and the branch target.
- as the branch is resolved in the ID stage, we only need to flush one instruction if the branch is taken. flush signal is triggered one cycle earlier ensuring the incorrect $PC+4$ instruction is replaced.
- If a register needed by the branch is being loaded by a lw instruction that is currently in EX stage, The pipeline

the instruction that is currently in Ex stage, the pipeline must stall for two cycles, because the branch needs the data in ID stage.

Question # 8 (Points: 15)

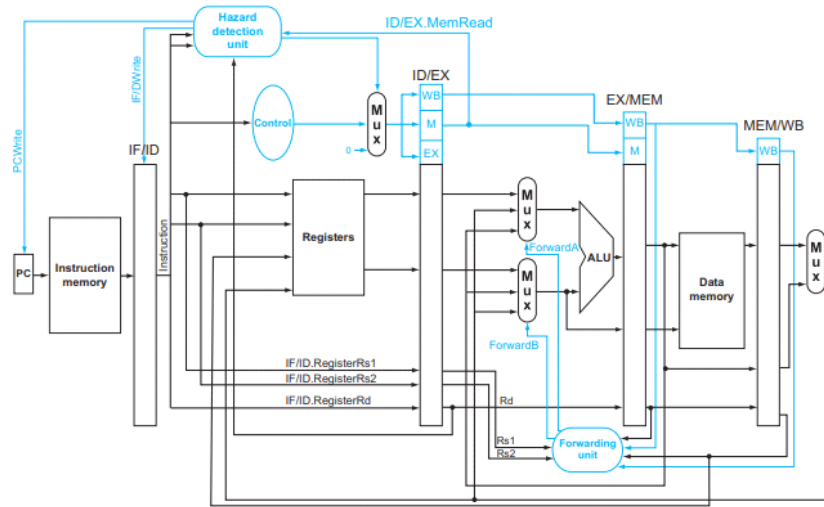
In a pipelined CPU, the multiply instruction (`mul`) requires two cycles in the EX stage (EXa and EXb).

1. Explain why this situation causes a structural hazard in the pipeline. Use a pipeline timing chart for a sequence containing two consecutive `mul` instructions to illustrate the conflict. (Assume there is no stall unit.) (6 marks)

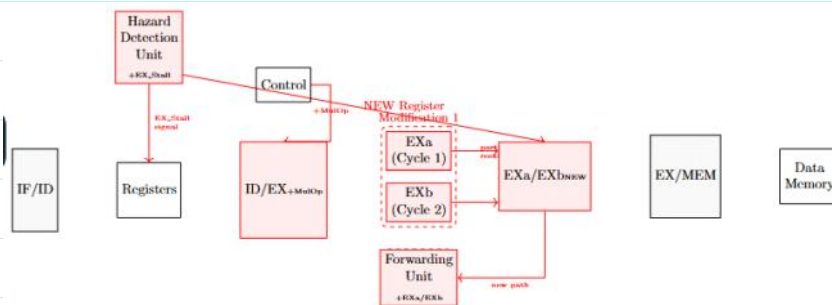


- Structural hazard occurs because both instructions use the same hardware simultaneously. In the fourth clock cycle, the `mul` instruction is in its second execution stage (EXb), while the next instruction is ready to enter its only execution stage (EX). Since there is only one ALU, a conflict occurs.

2. Clearly highlight or annotate the modifications needed to support a two-cycle `mul` instruction using Figure 4.58 from the textbook. Indicate any new control signals, hardware components, or changes to existing control logic that prevent this hazard. (6 marks)



- we need to divide EX stage into two stages, exa, exb.



- broke EX stage into two.
- we added pipeline register in between the major stages IF/ID, ID/EX, EX/MEM, MEM/WB
- The new pipeline is called EXa/EXb, between substages EXa and EXb.
- forwarding unit also receives EXa/EXb. Reg write, EXb/MEM. Reg write.

3. Briefly describe how your proposed modifications resolve the hazard. (3 marks)

Design Concept:

WE changed the pipeline by doubling the ALU hardware and splitting the execution into two phases: EXa and EXb.

A new control signal, MUL, tells the hardware how to go through. If (MUL=0) it means it's a normal instruction and the result finishes in EXa and just passes through EXb to reach Memory. If it's a multiply (MUL=1), both EXa and EXb work to finish the math before sending it to the MEM stage.

Forwarding:

Because we added a stage, the Forwarding Unit gets new rules to find the most latest values:

- **From EXa/EXb:** If the previous instruction was not a multiply (MUL=0), we grab the result here.
- **From EXb/MEM:** If the previous instruction was multiply (MUL=1), we have to wait until this stage to get the finished result.
- **From MEM/WB:** The standard path for older instructions.

The NEW Stall Rule:

If we try to use the result of a mul in the very next instruction, the result is still not finished. To fix this, the Hazard Detection Unit must treat this exactly like it is forcing a stall so the add waits for the mul to finish its extra execution cycle.